A Compositional JavaScript IR for CNC CAM

Geometry, machine actions, formal semantics, static guarantees, and G-code refinement

Design and reference implementation

2026-08-08

Executive summary

This report proposes and implements a layered JavaScript/TypeScript API for CNC CAM programs. The central design decision is to stop treating G-code text as the primary program representation. Geometry, process planning, canonical machine actions, analysis, simulation, and controller-specific G-code are separate semantic layers.

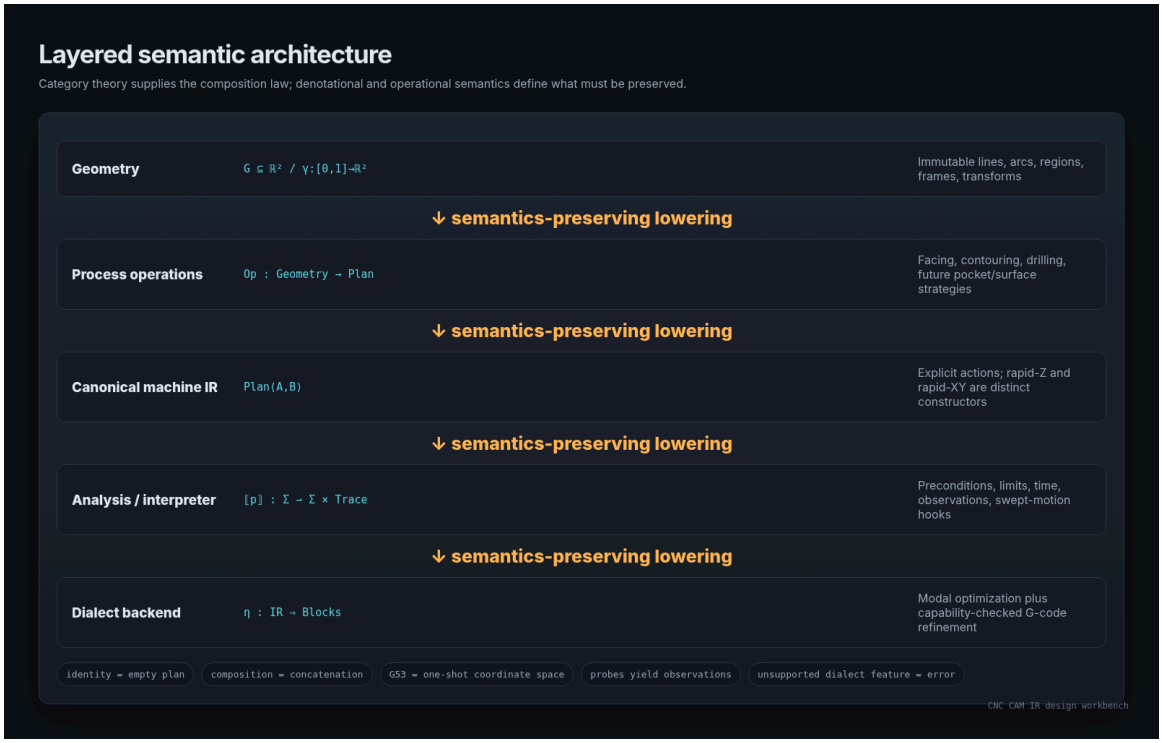
The uploaded Dropcut v3 prototype already contains the seed of such an architecture. It uses an internal move structure shaped like `{ kind, phase, pts, ops }`, with semantic distinctions between rapid, plunge, ramp, rough cut, and finish cut. It also contains useful downstream algorithms: center-form arc fitting in G17/G18/G19 and dexel material-removal verification. The problem is not lack of capability; it is that generation, linking, safety assumptions, timing, arc optimization, rendering, and G-code emission are still coupled inside one application.

The reference implementation introduces:

- immutable line/arc geometry and frame-aware points;
- branded units for millimetres, feed, spindle speed, dwell, and angles;
- a phase-indexed `Plan<A,B>` whose identity is the empty plan and whose composition is command concatenation;
- a fluent `Job` API for machine actions, facing, contouring, drilling, probing, pauses, coolant, spindle, work offsets, homing, and program end;
- a serializable canonical IR with a JSON Schema;
- a static analyzer for clearance, tool/spindle sequencing, limits, arc geometry, probe setup, and safe end states;
- an operational interpreter that produces motion traces, probe observations, distances, and time estimates;
- capability-checked G-code profiles, including G38-style and Fanuc-style probing distinctions;
- an adapter from the prototype’s current move representation;
- executable tests for algebraic laws and machine-safety invariants.

The strongest guarantee is structural rather than heuristic: the canonical IR has distinct `rapid-z` and `rapid-xy` constructors. A mixed-axis `G0 X... Y... Z...` block is therefore not representable by a conforming backend. This directly addresses the supplied warning that rapid moves may dogleg and should not be assumed to follow a coordinated straight line.

This is a design reference and working prototype, not a production-certified postprocessor. It cannot prove fixture clearance, controller conformance, tool suitability, stock setup, calibration, or physical collision freedom. Those require a machine-specific profile, swept-volume verification, dry runs, and operational controls.



Architecture overview

1. Deliverables

The package contains the following runnable artifacts:

Artifact	Purpose
`src/geometry.ts`	Immutable 2D line/arc geometry, points, transforms, bounds
`src/ir.ts`	Canonical commands, `Plan<A,B>`, program document
`src/dsl.ts`	Fluent `Job` API and basic milling operations
`src/analyze.ts`	Static preflight and diagnostics
`src/interpreter.ts`	Executable operational semantics and trace generation
`src/gcode.ts`	Modal, capability-checked G-code compiler
`src/legacy-dropcut.ts`	Adapter for `{kind, phase, pts, ops}`
`schema/cam-ir-v0.1.schema.json`	Machine-readable IR schema
`examples/badge.ts`	Facing, arc contouring, profile contour, tool change, peck drilling
`examples/probing.ts`	Probe observation and G38-style output sketch
`tests/*.test.mjs`	Eight executable tests
`docs/FORMAL_SEMANTICS.md`	Compact formal statement of the model
`artifacts/generated/badge.ir.json`	Serialized example IR
`artifacts/generated/badge.nc`	Compiled example G-code
`artifacts/generated/badge-summary.json`	Analysis and interpretation metrics
`artifacts/generated/probing.ir.json`	Serialized two-pass probing example
`artifacts/generated/probing.nc`	G38.2 fast/slow probe output

Run the implementation with:

```
npm run build
npm test
npm run example
npm run example:probe
```

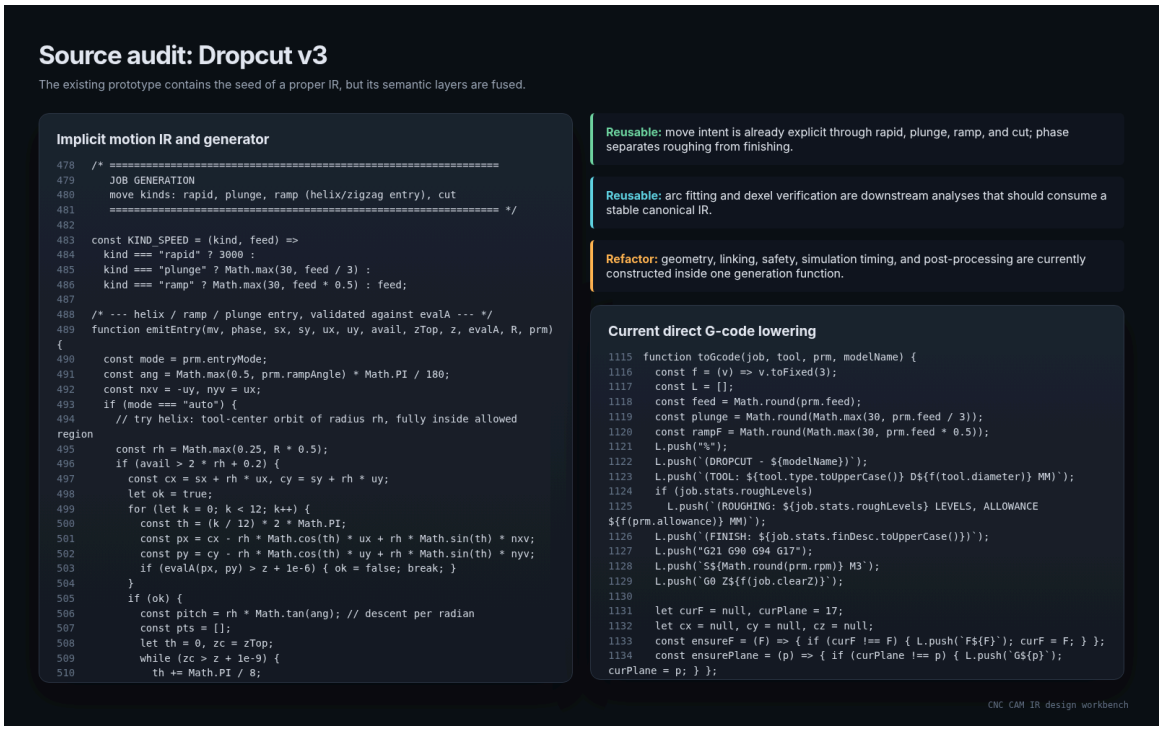
2. Audit of the uploaded Dropcut

prototype

2.1 The prototype already has an implicit IR

The prototype declares four move kinds – `rapid`, `plunge`, `ramp`, and `cut` – and adds a rough/finish phase. Each move contains a flat point array and may contain fitted line/arc operations. That is already more meaningful than a list of G-code blocks: it distinguishes process intent from syntax.

The source then flattens those moves into positions, kind tags, cumulative time, cut length, roughing time, and finishing time. The same representation drives viewport rendering, simulation, and G-code generation. This proves that a canonical motion layer is the correct seam for the application.



Source audit

2.2 Reusable components

The following parts should be retained as algorithms, but redirected to the new IR:

1. **Exact drop-cutter evaluation.** The spatial grid and ball/flat cutter contact calculations are geometry kernels. They should remain independent of G-code and UI state.

2. **CL-field construction.** The sampled cutter-location surface is a process-planning input.
3. **Marching squares and Eikonal finishing.** These produce geometric curves or tool-center paths.
4. **Entry validation.** Helix/ramp/plunge selection is a local planning pass with collision predicates.
5. **Arc fitting.** `compressCut` and `fitArcsRun` are backend-neutral path normalization passes.
6. **Dexel verification.** Material-removal simulation is an interpreter over canonical cutting motion.
7. **Three.js rendering.** Rendering should consume a trace or normalized motion IR, not generator internals.

2.3 Coupling that should be removed

The current `generateJob` function performs several semantically different jobs:

- chooses process strategies;
- samples geometry;
- links paths;
- decides rapid/retract behavior;
- assigns feed classes;
- estimates time;
- generates display buffers;
- prepares arc fitting inputs.

The current `toGcode` function then consumes the internal move objects directly. It establishes a basic preamble and modal plane/feed state, but it omits an explicit work offset, tool change, tool-length policy, coolant policy, and machine-coordinate initial retract. More importantly, rapid points are emitted as `G0 X... Y... Z...` blocks. That preserves the dogleg hazard identified in the supplied G-code notes.

The new architecture does not require rewriting the mathematical CAM kernels. It requires changing their output boundary.

2.4 Source-to-layer map

Existing prototype section	New layer
STL parsing, model normalization, spatial hash, exact contact	Geometry kernel
CL field, marching squares, Eikonal solve	Process-planning data and algorithms
Roughing/finishing strategy selection	Process operation compiler
`{kind, phase, pts, ops}`	Legacy motion IR, adapted to canonical commands
Arc fitting	Canonical path normalization
Time flattening and viewport lines	Operational interpreter / trace consumer
`toGcode`	Dialect backend
Dexel verification	Material-removal interpreter

3. Design goals

3.1 Primary goals

The API should be:

- **Composable.** Independent plans combine without knowing about G-code modal state.
- **Serializable.** The core IR is plain data, not closures or React state.
- **Analyzable.** Safety and capability checks run before text emission.
- **Machine-independent.** Internal motion uses canonical absolute millimetres and feed-per-minute semantics.
- **Dialect-aware at the edge.** G38 and G31 are not treated as interchangeable when their failure semantics differ.
- **Intent-preserving.** A plunge, ramp, link, cut, rapid, probe, dwell, pause, and tool change remain distinguishable.
- **Geometry-friendly.** Lines, arcs, contours, transforms, and future regions are first-class values.
- **Extensible.** Drop-cutter surface paths, dexel verification, cutter compensation, and multi-axis work can be added without changing the basic semantic split.

3.2 Non-goals for version 0.1

The implementation intentionally does not attempt:

- robust polygon offsets or general pocket clearing;
- NURBS, splines, or ellipse output;
- full controller macro languages;
- dynamic branches based on probe observations;
- cutter-radius or tool-length compensation;
- canned drilling cycles;
- five-axis inverse kinematics;
- fixture/holder collision proof;
- feed optimization from chip load, engagement, acceleration, or jerk;
- formal proof in a theorem prover.

These omissions keep the semantic core small enough to inspect and test.

4. Layered architecture

The architecture has five major layers.

4.1 Geometry

Geometry is pure and machine-independent. A contour contains a start point and a sequence of line or circular-arc segments. It denotes an oriented piecewise-circular curve in the work plane.

```
interface Contour2 {
  kind: "contour2";
  start: Point2;
  segments: readonly (Line2 | Arc2)[];
  closed: boolean;
}
```

Geometry supports translation, rotation, reflection, and uniform scale. A non-uniform affine transform maps circles to ellipses, so it is rejected when a contour contains arcs. This is an example of preserving the denotation rather than accepting a convenient but false type.

4.2 Process operations

A process operation turns geometry plus machining parameters into a canonical plan. Examples in version 0.1 are:

- `face(rect, options)`;
- `contour(curve, options)`;
- `drill(points, options)`.

The Dropcut roughing, hybrid finishing, constant-scallop finishing, and entry algorithms should become additional operation compilers that produce the same canonical command vocabulary.

4.3 Canonical machine IR

The canonical IR contains explicit semantic actions. It does not contain G-code words or inherited modal assumptions.

Representative commands are:

```
type Command =
  | { kind: "select-work-offset"; offset: "G54" | ... }
  | { kind: "tool-change"; tool: Tool }
  | { kind: "spindle"; mode: "off" | "cw" | "ccw"; speed?: RPM }
  | { kind: "coolant"; mode: "off" | "mist" | "flood" }
  | { kind: "rapid-z"; space: "work" | "machine"; z: Mm }
  | { kind: "rapid-xy"; space: "work" | "machine"; x: Mm; y: Mm }
  | { kind: "feed-line"; to: WorkPoint; feed: Feed; intent: Intent }
  | { kind: "feed-arc"; plane: Plane; to: WorkPoint; center: WorkPoint; ... }
  | { kind: "probe"; to: WorkPoint; onFailure: "error" | "continue"; capture: ObservationRef }
  | { kind: "dwell"; duration: Seconds }
  | { kind: "pause"; stop: "mandatory" | "optional" }
  | { kind: "home" }
  | { kind: "end" };
```

4.4 Analysis and interpretation

The analyzer executes a conservative abstract interpretation. It tracks enough state to reject common invalid programs without pretending to know the physical machine.

The interpreter executes the concrete operational model, producing a trace of rapid lines, feed lines, arcs, and probe moves, along with event records for tool changes, spindle, coolant, dwell, pause, home, and end.

4.5 Dialect backend

The G-code compiler is the only component that knows about:

- `G17/G18/G19` plane words;
- `G2/G3` direction conventions;

- `I/J/K` center offsets;
- `G38.2/.3/.4/.5` versus `G31`;
- `G53` one-shot machine coordinates;
- dwell `P` units;
- modal feed and plane suppression;
- `M3/M4/M5`, coolant, tool change, and program end syntax.

5. Mathematical model

5.1 Geometry denotation

A point is an element of Euclidean space. A contour denotes an oriented curve:

$$[\![g]\!] = \gamma : [0, 1] \to \mathbb{R}^2.$$

Each line segment is affine in its local parameter. Each arc segment lies on a circle and carries an orientation. The path denotation is obtained by piecewise concatenation with a reparameterization of the unit interval.

A similarity transform T acts on geometry by composition:

$$[\![T(g)]\!] = T \circ [\![g]\!].$$

For similarities, lines remain lines and circles remain circles. For orientation-reversing similarities, clockwise and counter-clockwise arc direction swaps. The implementation tests this law.

5.2 Plans as an indexed free category

Let the coarse phase objects be:

$$\mathscr{P} = \{\text{idle-no-tool},\ \text{idle-with-tool},\ \text{cut-ready},\ \text{ended}\}.$$

Primitive commands are typed generators between phase objects. A plan is a morphism:

$$\mathrm{Plan}(A, B) : A \to B.$$

The identity morphism is the empty sequence:

$$\mathrm{id}_A = [\,].$$

Composition is concatenation when the intermediate phase matches:

$$(q \circ p).\mathit{commands} = p.\mathit{commands} + + \, q.\mathit{commands}.$$

Identity and associativity hold by construction because array concatenation has those laws. The test suite executes both laws.

This phase index is intentionally coarse. It prevents obvious API misuse such as calling `contour` before starting the spindle in TypeScript. It does not encode exact position, clearance, tool identity, work offset, or machine limits at the type level. Those depend on values and are checked by the analyzer.

5.3 Denotational semantics of machine programs

Define machine state:

$$\Sigma = Position \times Tool \times Spindle \times Coolant \times WorkOffset \times Modal \times Observations.$$

A command denotes a partial state-and-trace transformer:

$$[\![c]\!] : \Sigma \rightharpoonup (\Sigma \times Trace).$$

Partiality represents an error: cutting with the spindle off, an invalid arc, unsupported probing semantics, an unsafe XY rapid, or an axis-limit violation.

A plan is the Kleisli composition of commands for the combined effects:

- `State<Σ>` for machine state;
- `Either<Diagnostic, _>` for failure;
- `Writer<Trace>` for observable motion and events.

The implementation uses ordinary TypeScript rather than an explicit monad library, but the semantic shape is the same.

5.4 Operational semantics

A small-step configuration is:

$$\langle pc, \sigma, \tau \rangle,$$

where `pc` is a command index, `σ` is machine state, and `τ` is accumulated trace.

Representative rules are:

Work-coordinate Z rapid

$$\frac{}{\langle pc, \sigma, \tau \rangle \rightarrow \langle pc + 1, \sigma[z_w := z], \tau \cdot RapidZ(z) \rangle}$$

The clearance predicate is updated from the resulting work Z.

XY rapid

$$\frac{\mathit{SafeZ}(\sigma)}{\langle pc, \sigma, \tau\rangle \rightarrow \langle pc+1, \sigma[x_w := x, y_w := y], \tau \cdot \mathit{RapidXY}(x,y)\rangle}$$

If `SafeZ(σ)` is false, the command is rejected.

Cutting line

$$\frac{\mathit{ToolLoaded}(\sigma) \quad \mathit{SpindleOn}(\sigma) \quad f > 0}{\langle pc, \sigma, \tau\rangle \rightarrow \langle pc+1, \sigma[p_w := p], \tau \cdot \mathit{FeedLine}(p,f)\rangle}$$

Probe

A probe command has an observation identifier `r` and an explicit failure policy. Its environment returns either a trip point or no trip:

$$\mathit{ProbeEnv}(\mathit{start}, \mathit{target}, \mathit{signal}) \in \mathit{Point} \cup \{\mathit{failure}\}.$$

The result binds `Observations[r]`. If the result is failure and policy is `error`, execution fails. This distinction is preserved into G38 variants and rejected for a plain G31 profile when automatic failure cannot be guaranteed.

5.5 G-code compilation as refinement

The compiler is a stateful transformation:

$$\eta_D : \mathit{CanonicalIR} \Rightarrow \mathit{GCode}_D$$

parameterized by dialect/profile `D`.

It may suppress redundant modal words, but this is allowed only when the interpreted target trace remains equivalent to the canonical trace within numeric tolerance.

The intended correctness condition is:

$$\mathit{Interpret}_D(\mathit{Compile}_D(p)) \approx [\![p]\!].$$

Version 0.1 does not include a full target G-code parser, so this condition is tested for selected invariants rather than proved globally. A future conformance harness should parse emitted G-code back into canonical traces and compare endpoints, planes, arc centers, probe policies, and modal state.

6. Why the rapid algebra matters

The supplied G-code notes emphasize that `G0` may move axes independently and can follow a dogleg path. A common API mistake is to expose only:

```
rapidTo({ x, y, z })
```

That API cannot distinguish safe intent from unsafe coincidence. A backend may emit a single `G0 X Y Z`, and a caller may incorrectly assume a coordinated line.

The canonical IR instead exposes:

```
{ kind: "rapid-z", ... }
{ kind: "rapid-xy", ... }
```

A safe high-level move expands to:

```
rapid-z(clearance)
rapid-xy(x, y)
rapid-z(targetZ)
```

The property is stronger than a linter rule. A conforming compiler cannot encounter a canonical mixed-axis rapid because no such constructor exists. Tests scan all generated `G0` blocks and require that none contain both XY and Z words.

Machine-coordinate retraction is equally explicit:

```
{ kind: "rapid-z", space: "machine", z: mm(-5) }
```

The backend lowers this to:

```
G53 G0 Z-5.000
```

`G53` is represented as a property of that command, not as modal state. This mirrors its one-shot semantics and prevents accidental leakage.

7. JavaScript and TypeScript API

7.1 Branded quantities

At runtime, quantities are numbers. In TypeScript, brands prevent accidental mixing:

```
type Millimetres = number & Brand<"mm">;
type MillimetresPerMinute = number & Brand<"mm/min">;
type RevolutionsPerMinute = number & Brand<"rpm">;
```

Constructors validate finiteness and sign:

```
mm(5)
mmPerMin(450)
rpm(12_000)
seconds(1.5)
```

The canonical representation is always millimetres and millimetres per minute. Output-unit conversion is a backend concern.

7.2 Coordinate frames

Points carry a frame parameter:

```
Point3<"work">
Point3<"machine">
```

Geometry lives in a work/part frame. `G53` actions use the machine frame. Version 0.1 does not model the full affine relationship between machine and work coordinates because the active work offset is controller state. It therefore avoids pretending that a machine-coordinate point can be freely subtracted from a work-coordinate point.

7.3 Geometry construction

```
const outline = roundedRectangle(
  point2(-30, -18),
  point2(30, 18),
  mm(3),
  "badge outline",
);

const emblem = circle(point2(0, 0), mm(8), "ccw");
```

A general contour builder supports line and center-form arc construction:

```
const path = contour(point2(0, 0), "slot")
  .lineTo(point2(20, 0))
  .arcTo(point2(25, 5), point2(20, 5), "ccw")
  .lineTo(point2(25, 15))
  .build();
```

7.4 Fluent machine program

```
const program = Job.begin(
  { name: "Badge" },
  {
    workClearanceZ: mm(5),
    machineSafeZ: mm(-5),
    arcTolerance: mm(0.01),
    requireMachineRetractBeforeToolChange: true,
  },
)
  .machineRapidZ(mm(-5))
  .selectWorkOffset("G54")
  .changeTool(flatEndMill(1, mm(3)))
  .spindleCW(rpm(12_000))
  .contour(emblem, {
    topZ: mm(0),
    bottomZ: mm(-1.2),
    stepDown: mm(0.6),
    feed: mmPerMin(450),
    plungeFeed: mmPerMin(150),
  })
  .machineRapidZ(mm(-5))
  .spindleOff()
  .end();
```

The chain’s type changes from `idle-no-tool` to `idle-with-tool`, then `cut-ready`, then back to `idle-with-tool`, then to a completed document. JavaScript callers receive the same runtime API but rely on `analyze()` for sequencing errors.

7.5 Composing algorithmic plans

The lower-level `Plan<A,B>` is useful when a strategy compiler generates commands:

```
const surfacePlan: Plan<"cut-ready", "cut-ready"> =
  Plan.fromCommands(generatedCommands);

const program = job.appendPlan(surfacePlan);
```

The Dropcut adapter returns exactly this type.

8. Canonical IR and serialization

A program document is immutable plain data:

```
{
  "schema": "https://example.invalid/cam-ir/v0.1",
  "version": "0.1",
  "metadata": { "name": "Compositional badge" },
  "safety": {
    "workClearanceZ": 5,
    "machineSafeZ": -5,
    "arcTolerance": 0.01,
    "requireMachineRetractBeforeToolChange": true
  },
  "commands": [
    { "kind": "rapid-z", "space": "machine", "z": -5 },
    { "kind": "select-work-offset", "offset": "G54" },
    { "kind": "tool-change", "tool": { "number": 1, "name": "3 mm flat end mill", "kind": "flat-end-
      mill", "diameter": 3 } }
  ]
}
```

The included JSON Schema validates the generated badge IR under draft 2020-12.

Serialization matters for several reasons:

- jobs can be cached and diffed;
- a worker or server can run planning separately from UI code;
- verification can operate on stored programs;
- postprocessors can be versioned independently;
- source maps and provenance can be attached;
- regression tests can use golden IR fixtures rather than fragile G-code text.

9. Static analysis

The analyzer tracks:

- loaded tool;
- spindle state;
- coolant state;
- selected work offset;
- known work position;
- known machine axes;
- safe-Z predicate;
- program-ended state;
- work bounds;
- counts of rapid, cutting, arc, tool-change, and probe commands.

9.1 Representative diagnostics

Code	Condition
`TOOL_CHANGE_WITH_SPINDLE`	Tool change while spindle is running
`TOOL_CHANGE_NOT_RETRACTED`	Tool change without established safe Z
`TOOL_CHANGE_NEEDS_MACHINE_Z`	Policy requires the configured G53 retract
`SPINDLE_WITHOUT_TOOL`	Spindle start without a loaded tool
`RAPID_XY_BELOW_CLEARANCE`	XY rapid without safe-Z predicate
`CUT_WITHOUT_TOOL`	Cutting/plunge/ramp without a tool
`CUT_WITH_SPINDLE_OFF`	Cutting/plunge/ramp while spindle is stopped
`ARC_UNKNOWN_START`	Arc emitted without a known start point
`ARC_RADIUS_MISMATCH`	Start and end radii differ beyond tolerance
`ARC_NON_PLANAR`	Arc changes its orthogonal coordinate
`PROBE_WITH_SPINDLE`	Probe command while spindle is on
`ZERO_LENGTH_PROBE`	Probe target equals start point
`HOME_NOT_RETRACTED`	Reference return without prior safe retract
`END_WITH_SPINDLE`	Program end while spindle is running
`END_NOT_RETRACTED`	Program end below safe Z
`AXIS_LIMIT`	Coordinate exceeds supplied machine envelope

Warnings include implicit work offset, non-probe tool used for probing, coolant left on at end, and similar conditions that may be intentional but deserve review.

9.2 Abstract interpretation, not physical proof

The analyzer is intentionally conservative and symbolic. It does not know fixture meshes, holder geometry, acceleration, servo following error, stock state, or controller bugs. It establishes semantic preconditions for the supported IR. Physical verification belongs in a later swept-volume or dexel interpreter.

10. Operational interpreter

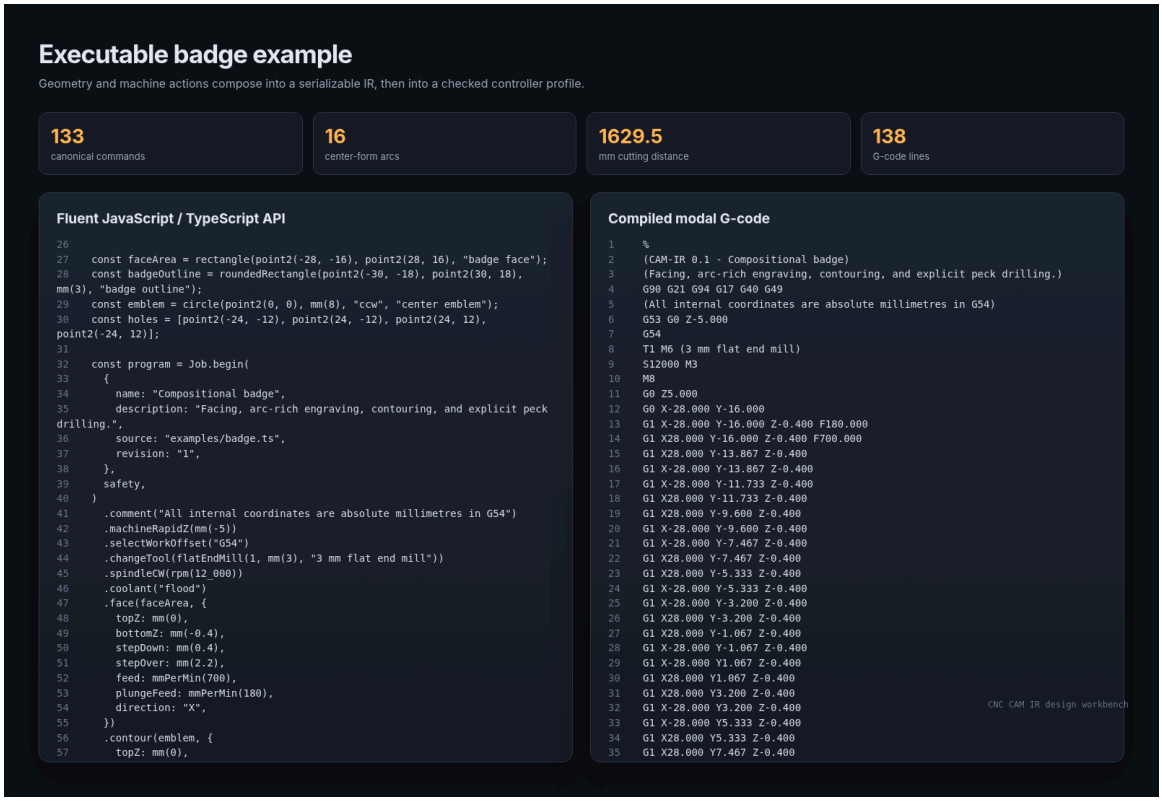
The interpreter consumes the same program document and produces:

- motion segments with command indices;
- event records;
- probe observation bindings;
- total distance;
- cutting distance;
- estimated time.

For lines, duration is length divided by feed. For arcs, it computes the directed sweep in the selected plane and multiplies by radius. Rapids use separate XY and Z profile rates, matching the structural decomposition of rapid motion.

The example result is:

Metric	Value
Canonical commands	133
Arc commands	16
Tool changes	2
G-code lines	138
Cutting distance	1629.5 mm
Estimated duration	203.45 s
Analyzer warnings	0



Executable example

10.1 Dexel verification integration

The prototype's `verifyJob` already interprets non-rapid segments as swept cutter footprints over a height field. The clean integration is:

```
ProgramDocument
  -> interpret/normalize motion trace
  -> sweep tool geometry through each cutting segment
  -> update dexel stock
  -> compare stock against target
  -> emit diagnostics and heatmap
```

This makes dexel verification independent of whether the path came from raster, hybrid waterline, constant scallop, imported G-code, or a future pocketing strategy.

Arc commands should either be swept analytically or adaptively subdivided under a verified chord tolerance. The current prototype already has the needed tolerance concepts.

11. G-code backend

11.1 Explicit preamble

The sample backend emits:

```
G90 G21 G94 G17 G40 G49
```

The exact preamble belongs to a dialect profile. A production profile should specify which reset codes are supported and whether tool-length compensation is managed externally.

11.2 Modal state

The compiler tracks the active plane and feed. It emits a new plane only when required and emits a feed word only when it changes. This optimization is local and semantics-preserving.

Canonical commands do not inherit modal state. For example, every feed command carries its feed semantically even if the target text omits a redundant `F` word.

11.3 Arc planes and direction

The IR defines arc direction mathematically in the listed coordinate pair:

- XY uses `(X,Y)`;
- XZ uses `(X,Z)`;
- YZ uses `(Y,Z)`.

Because `(X,Z)` has the opposite orientation from the usual +Y view, a mathematical XZ counter-clockwise arc maps to `G2`, while XY and YZ counter-clockwise arcs map to `G3`. This matches the orientation handling already present in the uploaded prototype.

Center form is always used:

```
G3 X-8.000 Y0.000 I-8.000 J0.000
```

Full circles are split into two semicircles for controller portability.

11.4 Probing capability checks

For a G38-style dialect:

IR signal	Failure policy	Output
contact	error	`G38.2`
contact	continue	`G38.3`
loss	error	`G38.4`
loss	continue	`G38.5`

For a plain Fanuc-style profile, the backend emits `G31` only when the requested semantics can be represented. An IR probe with `onFailure: "error"` is rejected because plain `G31` requires a controller-specific macro/status check to guarantee that behavior.

This is an important design rule: unsupported semantics are compilation errors, not comments or silent approximations.

11.5 Source maps

The compiler records the G-code line range produced by each canonical command. This supports:

- error reporting;
- UI selection from toolpath to G-code;
- simulation synchronization;
- postprocessor debugging;
- audit trails.

12. Migration plan for Dropcut v3

Phase 1: Introduce the canonical boundary

Keep `generateJob` and its algorithms, but replace the final `moves` array with `Plan<"cut-ready","cut-ready">` or adapt it using `adaptDropcutMoves`.

The included adapter:

- maps rapid/plunge/ramp/cut intent;

- maps fitted G17/G18/G19 arcs;
- expands legacy rapid points through explicit clearance;
- returns a typed plan that can be appended to a job.

Phase 2: Move G-code generation out of React

Replace `toGcode(job, tool, prm, modelName)` with:

```
const analysis = analyze(program, machineProfile.constraints);
const output = compileGCode(program, machineProfile);
```

The React component should render diagnostics and download `output.text`; it should not own modal state.

Phase 3: Redirect simulation and rendering

Use `interpret(program)` to generate a trace. Render the trace in Three.js and drive the DRO from trace time. This eliminates the bespoke flattened arrays as a second source of truth.

Phase 4: Redirect dexel verification

Make the verifier consume trace segments. Add analytical or tolerance-bounded arc sweeping.

Phase 5: Refactor strategy outputs

Change roughing, raster, hybrid waterline, and constant-scallop functions to return geometry/path values first, then lower them through shared linking and entry passes. This removes duplicated cursor and retract logic.

Phase 6: Add target-specific profiles

Create profiles for the actual Makera controller and any LinuxCNC, GRBL, Mach, Haas, or Fanuc targets. Each profile should declare:

- supported G/M codes;
- probe dialect and result mechanism;
- dwell units;

- arc-center mode;
- supported planes;
- precision;
- work/tool offset policy;
- machine envelope;
- safe machine Z;
- spindle/feed limits;
- tool-change behavior;
- program-end behavior.

13. Guarantees and their boundaries

13.1 Compile-time guarantees for TypeScript clients

- branded quantities reduce unit confusion;
- work and machine points are distinct types;
- `Job` methods enforce coarse phase sequencing;
- `Plan<A,B>` composition requires matching phase indices;
- geometry unions require exhaustive handling.

13.2 Structural guarantees in the IR

- no mixed-axis rapid constructor;
- G53 is explicit and one-shot;
- feed moves carry feed semantically;
- arcs carry plane, center, direction, and endpoint;
- probes carry signal, failure policy, and observation identifier;
- program end is explicit;
- IR is serializable and versioned.

13.3 Static analyzer guarantees

Subject to its abstract model, the analyzer checks:

- tool/spindle prerequisites;
- safe XY rapid predicate;

- machine retract before tool change when configured;
- feed and spindle limits;
- arc radius and planarity;
- probe prerequisites;
- safe home and end state;
- supplied axis envelopes.

13.4 Backend guarantees

- unsupported arc planes fail;
- unsupported probes fail;
- G31 cannot masquerade as automatic G38.2 failure behavior;
- center offsets are emitted from the canonical start point;
- rapid XY and Z remain separate blocks;
- output-unit conversion is centralized.

13.5 Not guaranteed

The implementation does **not** guarantee:

- collision freedom with stock, clamps, fixtures, spindle, holder, or machine;
- that configured safe Z is physically safe;
- that the controller implements its documented dialect correctly;
- that a tool, feed, speed, stepdown, stepover, or entry is physically appropriate;
- that arc interpolation matches the assumed center/direction convention on every controller;
- that machine limits, work offsets, tool offsets, and homing are calibrated;
- that numeric rounding is acceptable for a specific tolerance;
- that the example G-code is safe to run on any real machine.

A production workflow still requires machine-profile validation, simulation, single-block proving, and operator procedures.

14. Test and validation results

The test suite currently has eight passing tests:

1. legacy Dropcut rapid adaptation stages motion through clearance;
2. `Plan` identity and associativity;
3. transform composition is functorial on points;
4. reflection reverses arc orientation;
5. cutting before setup is rejected;

6. no compiled `G0` block contains XY and Z together;
7. a circle lowers to two center-form arcs;
8. Fanuc G31 rejects automatic no-contact failure semantics.

The generated JSON IR also validates against the included draft 2020-12 JSON Schema.

Test output

15. Recommended next extensions

15.1 Region algebra and robust offsets

The geometry layer should grow from contours to oriented regions with holes:

$$Region2 = OuterBoundary \setminus \bigcup Holes.$$

Required operations include union, intersection, difference, Minkowski sum, tool-radius offset, and medial-axis or skeleton queries. These should use a robust computational-geometry kernel rather than ad hoc floating-point clipping.

15.2 Strategy interface

Define a strategy protocol:

```
interface Strategy<Input, Output extends GeometryOrPath> {
  plan(input: Input, context: PlanningContext): Result<Output, Diagnostic[]>;
}
```

Drop-cutter raster, hybrid waterline, constant-scallop, roughing, and entry should implement this protocol. Strategy output should be geometry/path data, not machine actions, until a shared linker and entry planner lowers it.

15.3 Observation expressions and control flow

Probe results eventually need first-class expressions:

```
const top = probe(...);
setWorkOffsetZ(top.z - probeLength);
```

A serializable design should avoid JavaScript closures. A suitable next IR is SSA-like:

```
%hit = probe ...
%z0  = sub (axis %hit Z) 12.345
set-work-offset G54 Z %z0
assert probe-succeeded %hit
```

Structured branches can lower to controller macros when supported or execute in a host-side streaming interpreter. Capability effects should make the required execution model explicit.

15.4 Refinement certificates

Each lowering pass can emit a certificate:

- source path identifier;
- maximum chord error;
- arc-fit residual;
- bounding box;
- minimum clearance sample;
- machine-profile hash;
- compiler version;
- source-map digest.

This would make generated G-code auditable and enable deterministic regression comparisons.

15.5 Target re-interpretation

Add a parser/interpreter for the emitted G-code subset. For every test program:

1. interpret canonical IR;
2. compile G-code;
3. parse and interpret target G-code;
4. compare traces under tolerance.

This is the most valuable next correctness test because it directly checks semantic preservation across modal compilation.

15.6 Physical verification

Unify the prototype’s dexel verifier with:

- holder and shank geometry;
- fixture meshes;
- machine-axis limits;
- rapid-path sweep;
- adaptive arc sweep;
- stock evolution;
- gouge/excess certificates.

The result should attach diagnostics to canonical command indices and therefore to G-code source-map lines.

16. Practical recommendations

The implementation suggests the following priorities:

1. Adopt the canonical command IR immediately and make React consume it rather than own it.
2. Preserve the current drop-cutter, arc-fitting, and dexel algorithms as independent passes.
3. Make safe rapid movement structural, not a convention.
4. Treat machine profiles as versioned configuration and reject unsupported semantics.
5. Keep geometry and machine actions separate until the process-operation lowering boundary.
6. Add G-code re-interpretation before expanding the command vocabulary.
7. Add robust region/offset geometry before implementing general pockets.
8. Add probe SSA/control flow only after deciding whether execution is controller-side or host-streamed.

17. File map

```
cnc-cam-ir/
├── README.md
├── package.json
├── tsconfig.json
├── src/
│   ├── units.ts
│   ├── geometry.ts
│   ├── ir.ts
│   ├── dsl.ts
│   ├── analyze.ts
│   ├── interpreter.ts
│   ├── gcode.ts
│   ├── legacy-dropcut.ts
│   └── index.ts
├── schema/
│   └── cam-ir-v0.1.schema.json
├── examples/
│   ├── badge.ts
│   └── probing.ts
├── tests/
│   ├── category.test.mjs
│   ├── safety.test.mjs
│   └── adapter.test.mjs
├── docs/
│   └── FORMAL_SEMANTICS.md
├── artifacts/
│   ├── generated/
│   │   ├── badge.ir.json
│   │   ├── badge.nc
│   │   ├── badge-summary.json
│   │   ├── probing.ir.json
│   │   └── probing.nc
│   └── screenshots/
└── report/
    ├── CNC_CAM_IR_Design_Report.md
    └── CNC_CAM_IR_Design_Report.pdf
```

Appendix A. Example G-code excerpt

```
%
(CAM-IR 0.1 - Compositional badge)
G90 G21 G94 G17 G40 G49
G53 G0 Z-5.000
G54
T1 M6 (3 mm flat end mill)
S12000 M3
M8
G0 Z5.000
G0 X-28.000 Y-16.000
G1 X-28.000 Y-16.000 Z-0.400 F180.000
G1 X28.000 Y-16.000 Z-0.400 F700.000
...
G3 X-8.000 Y0.000 I-8.000 J0.000 F450.000
G3 X8.000 Y0.000 I8.000 J0.000
...
G53 G0 Z-5.000
M5
M30
%
```

The example demonstrates the intended skeleton, but its numeric parameters and generic profile are illustrative. They are not authorization to run the file on a real machine.

Appendix B. Proposed invariant checklist

A production compiler should refuse output unless all required checks pass:

- [ ] program schema/version supported;
- [ ] explicit output units and distance/feed modes;
- [ ] explicit work offset policy;
- [ ] machine-safe initial retract;
- [ ] tool change only with spindle stopped and safe retract;
- [ ] spindle speed within profile;
- [ ] feed within profile;
- [ ] every cutting move has tool and spindle prerequisites;
- [ ] every XY rapid has a proven safe-Z predicate;
- [ ] no mixed-axis rapid block;
- [ ] every arc has known start, matching radius, valid plane, and supported dialect;
- [ ] probe failure semantics supported by target;

- [ ] home only after safe retract;
- [ ] coolant and spindle stopped before end;
- [ ] final safe retract;
- [ ] bounds and fixture verification completed;
- [ ] compiled G-code re-interprets to an equivalent trace;
- [ ] machine-specific dry run completed.